

Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch

```
C:\Users\johnf\Downloads\ggs\my_first_flutter_app>git branch
feature/data-ui
* feature/student-interactions
feature/student-list
feature/widgets
main
```

🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using `onPressed`.

Right now your buttons are only decorations.

Research how Flutter's `onPressed` works.

The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:

- Student Card with buttons
- `onPressed` inside your code
- Button reacting or terminal message

```

292 ElevatedButton.icon(
293   onPressed: () {
294     ScaffoldMessenger.of(context).showSnackBar(
295       SnackBar(
296         content: Text(
297           "Edit button pressed for ${student.name}",
298         ), Text
299       ), SnackBar
300     );
301   },
302   icon: const Icon(Icons.edit),
303   label: const Text("Edit"),

```

Student Directory

Student Directory



Ivan Villareal
BSIT - 3rd Year
Age: 24
Hobby: Photography, Videography, & Video Editing

Student ID: 2026-0001
Email: ivan.villareal@dbtc-cebu.edu.ph
Favorite Subject: Snacks

Favorite

Edit



Ivan Villareal
BSIT - 3rd Year
Age: 24
Hobby: Photography, Videography, & Video Editing

Student ID: 2026-0001
Email: ivan.villareal@dbtc-cebu.edu.ph
Favorite Subject: Snacks

Favorite

Edit



John Kevin Villacorte
BSIT - 4th Year
Age: 23
Hobby: Drawing

Student ID: 2026-0004
Email: john.villacorte@dbtc-cebu.edu.ph
Favorite Subject: Web Development

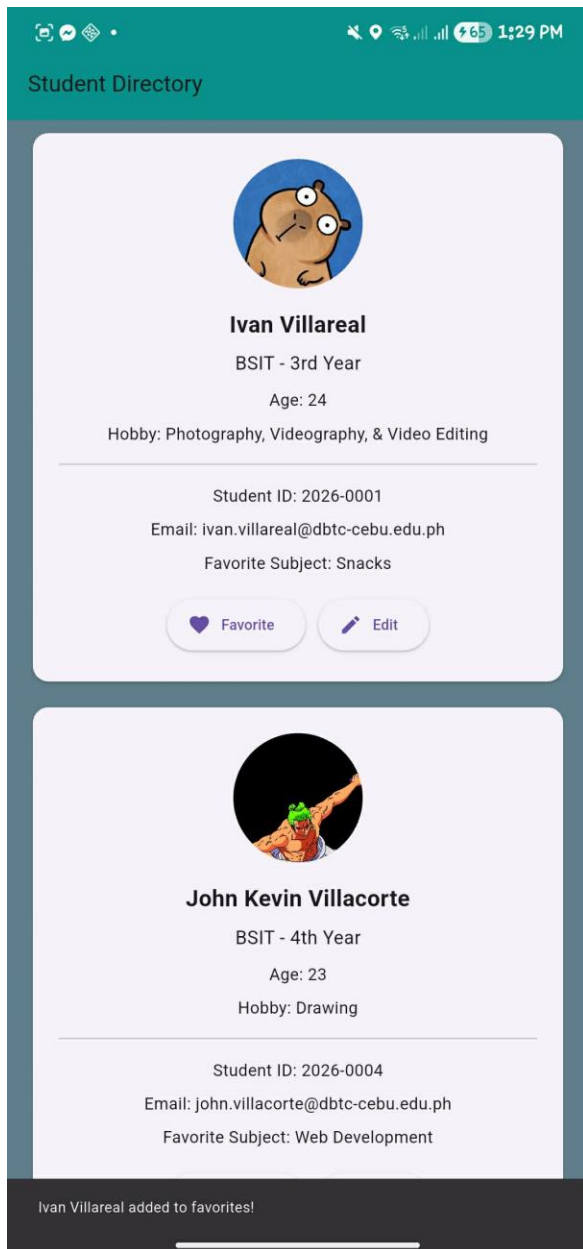


John Kevin Villacorte
BSIT - 4th Year
Age: 23
Hobby: Drawing

Student ID: 2026-0004
Email: john.villacorte@dbtc-cebu.edu.ph
Favorite Subject: Web Development

Ivan Villareal added to favorites!

Edit button pressed for Ivan Villareal



🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

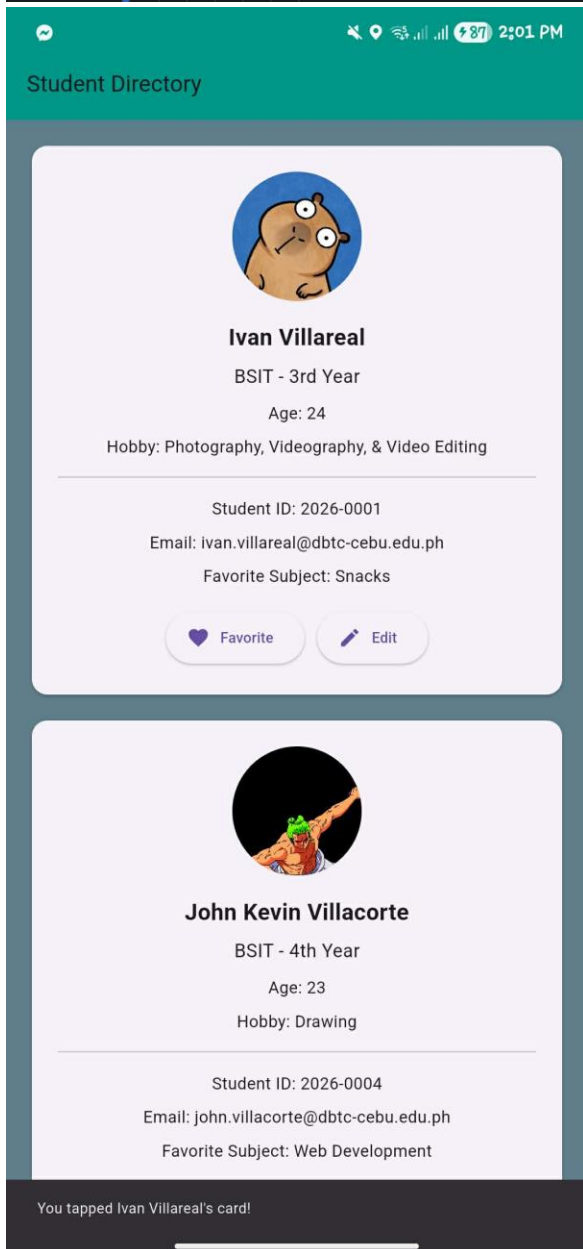
Screenshot:

- `onTap` in your code
- Student Card being tapped

```

168         return InkWell(
169             onTap: () {
170                 ScaffoldMessenger.of(context).showSnackBar(
171                     SnackBar(
172                         content: Text(
173                             "You tapped ${student.name}'s card!",
174                         ), Text
175                     ), SnackBar
176                 );
177             },

```



🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

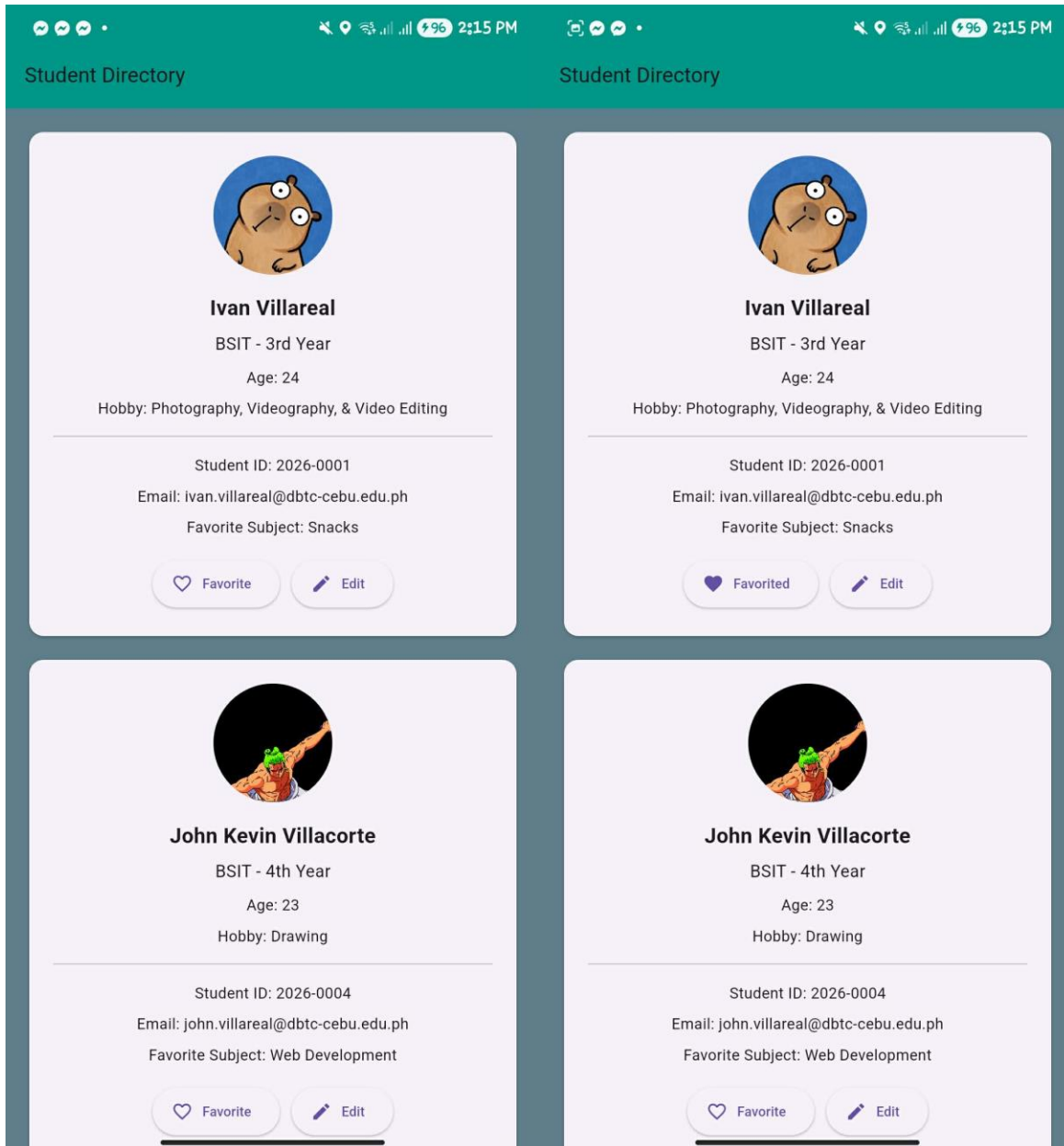
Test

The screen should visibly change immediately after pressing the button.

Screenshot:

- `setState` in your code
- Before and after interaction

```
287 setState(() {
288   isFavorite[index] =
289     !isFavorite[index];
290 });
291 },
292 icon: Icon(
293   isFavorite[index]
294     ? Icons.favorite
295     : Icons.favorite_border,
296 ), icon
297 label: Text(
298   isFavorite[index]
299     ? "Favorited"
```

🚩 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

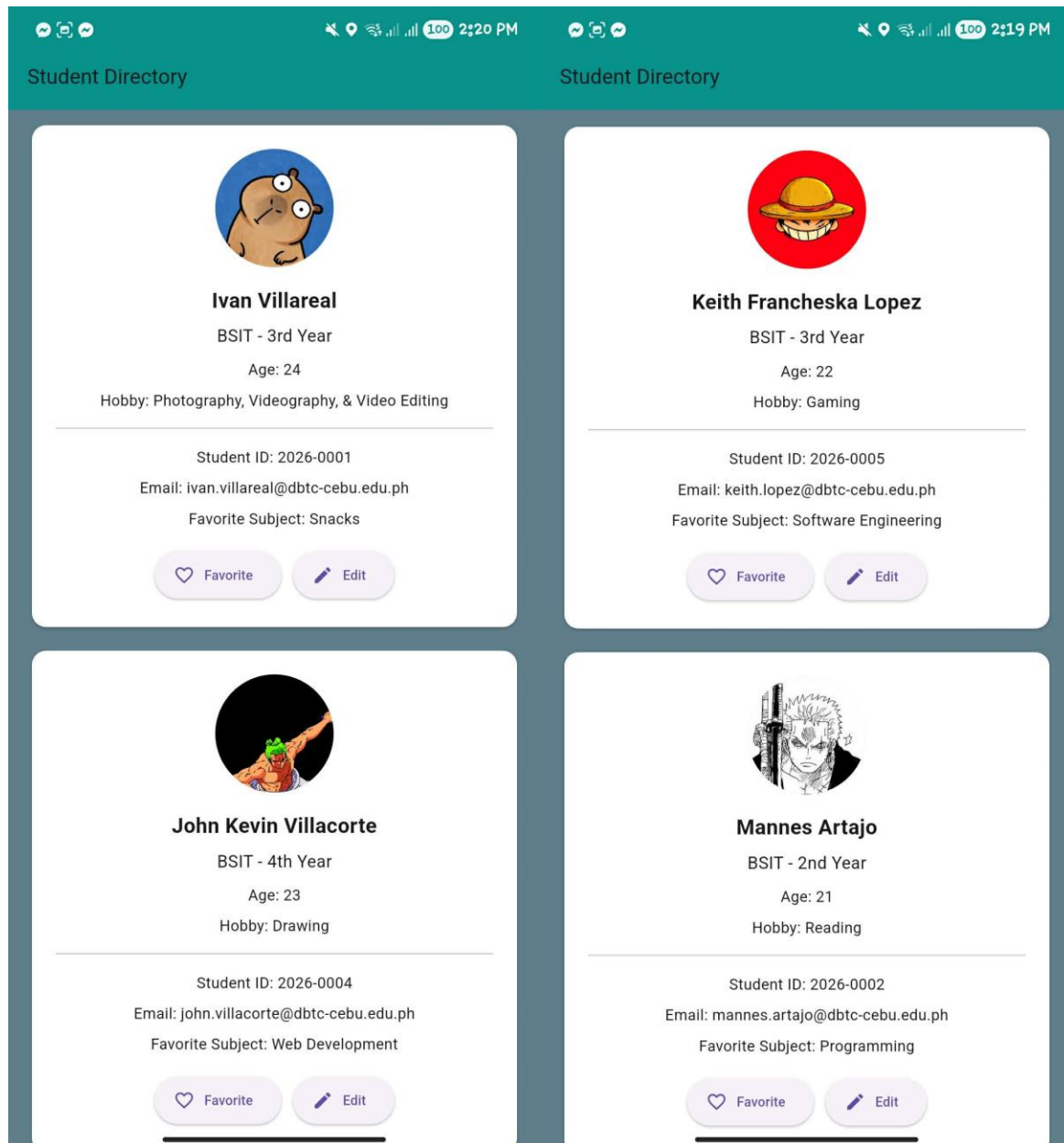
Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:

- Favorited student
- Non-favorited student
- Favorite icon changing



Ivan Villareal

BSIT - 3rd Year

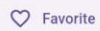
Age: 24

Hobby: Photography, Videography, & Video Editing

Student ID: 2026-0001

Email: ivan.villareal@dbtc-cebu.edu.ph

Favorite Subject: Snacks



Favorite



Edit



Keith Francheska Lopez

BSIT - 3rd Year

Age: 22

Hobby: Gaming

Student ID: 2026-0005

Email: keith.lopez@dbtc-cebu.edu.ph

Favorite Subject: Software Engineering



Favorite



Edit



John Kevin Villacorte

BSIT - 4th Year

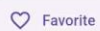
Age: 23

Hobby: Drawing

Student ID: 2026-0004

Email: john.villacorte@dbtc-cebu.edu.ph

Favorite Subject: Web Development



Favorite



Edit



Mannes Artajo

BSIT - 2nd Year

Age: 21

Hobby: Reading

Student ID: 2026-0002

Email: mannes.artajo@dbtc-cebu.edu.ph

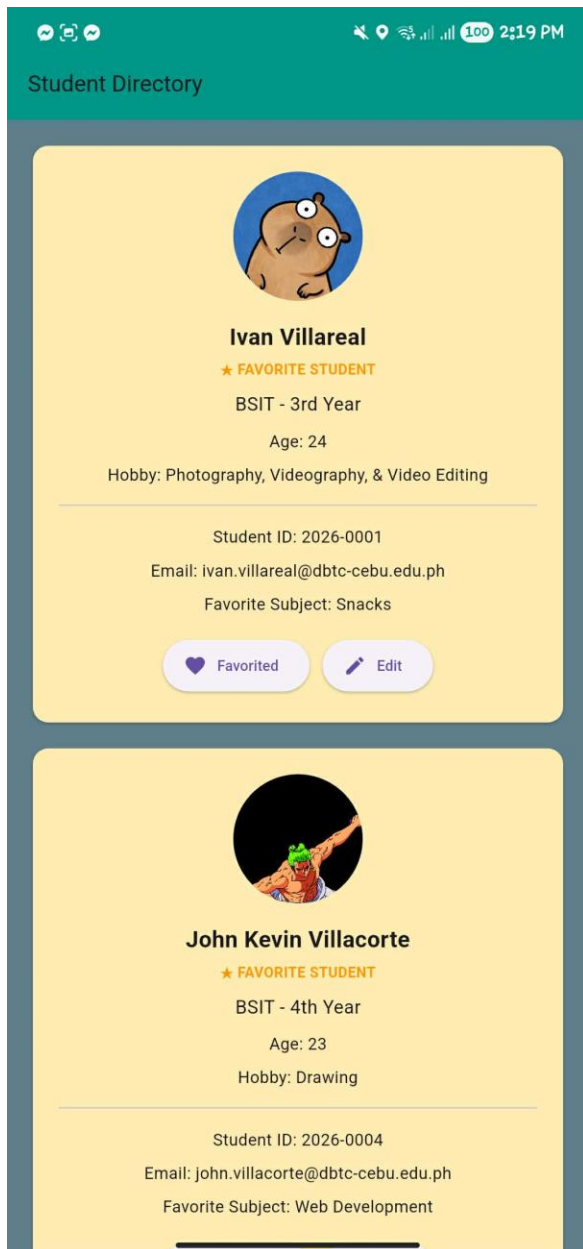
Favorite Subject: Programming



Favorite



Edit



🚩 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart [List](#).

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B
- Student C

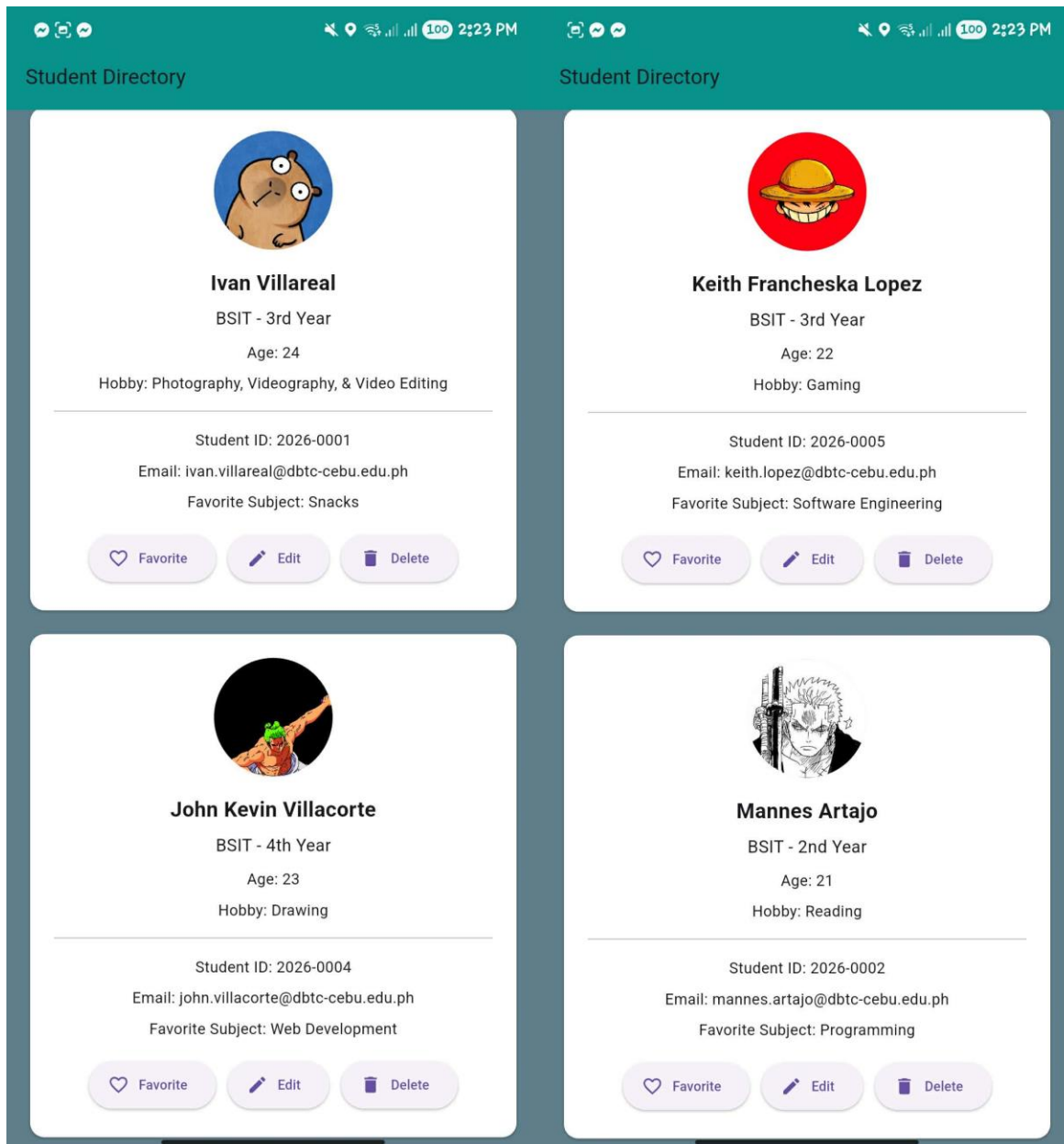
After deleting Student B:

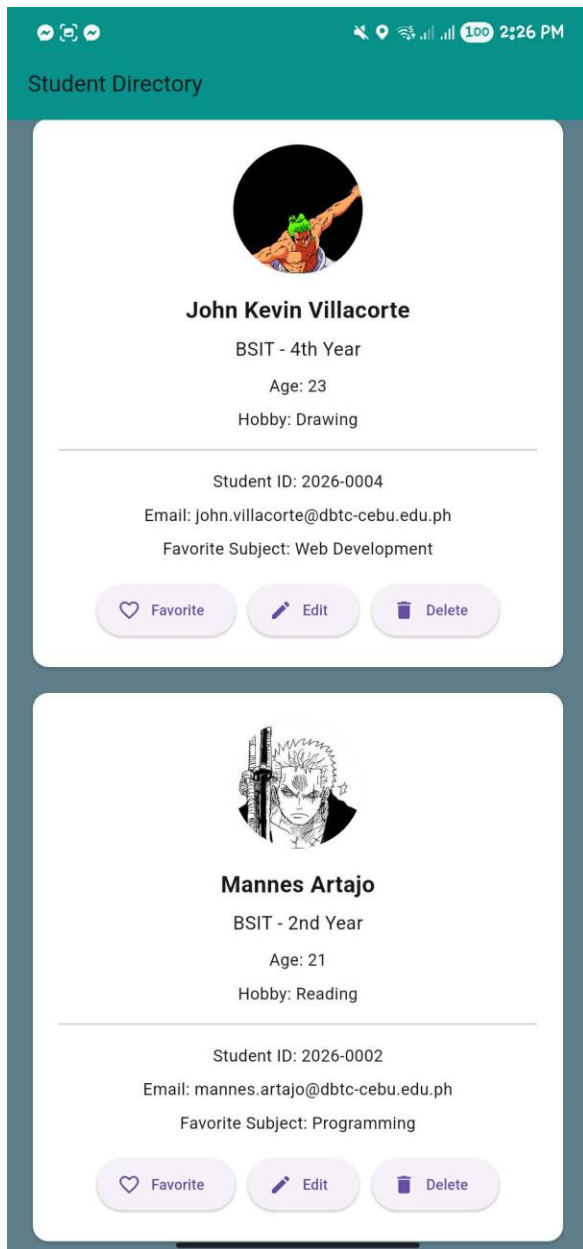
- Student A
- Student C

No empty spaces should remain.

Screenshot:

- Student before deletion
- Student after deletion





🚩 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`
- `SnackBar`
- Bottom Sheet

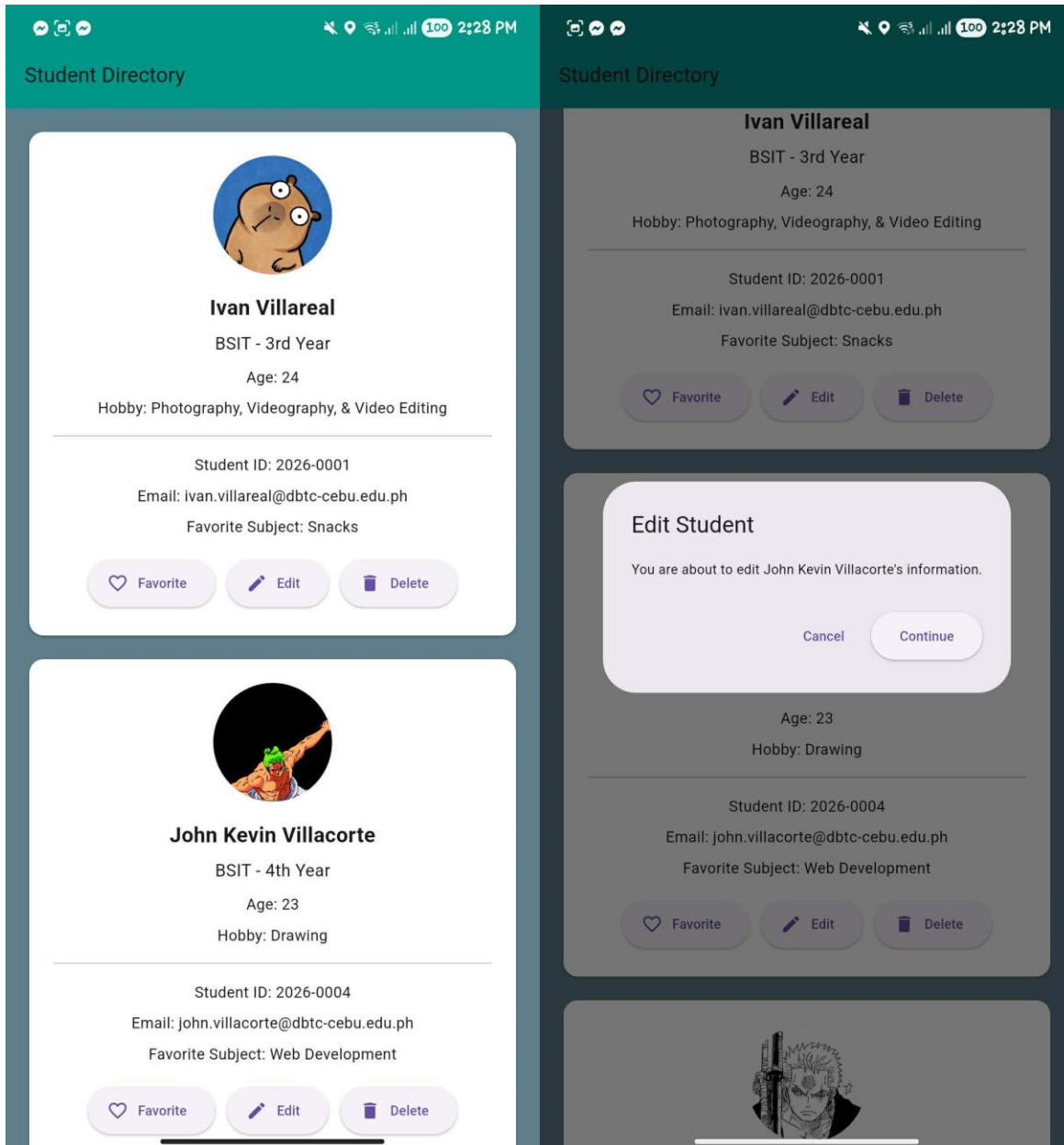
The goal is simply to launch an edit action, not build the complete editor yet.

Test

Pressing Edit should open your chosen interface.

Screenshot:

- Edit button
- Opened dialog, snackbar, or bottom sheet



🚩 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

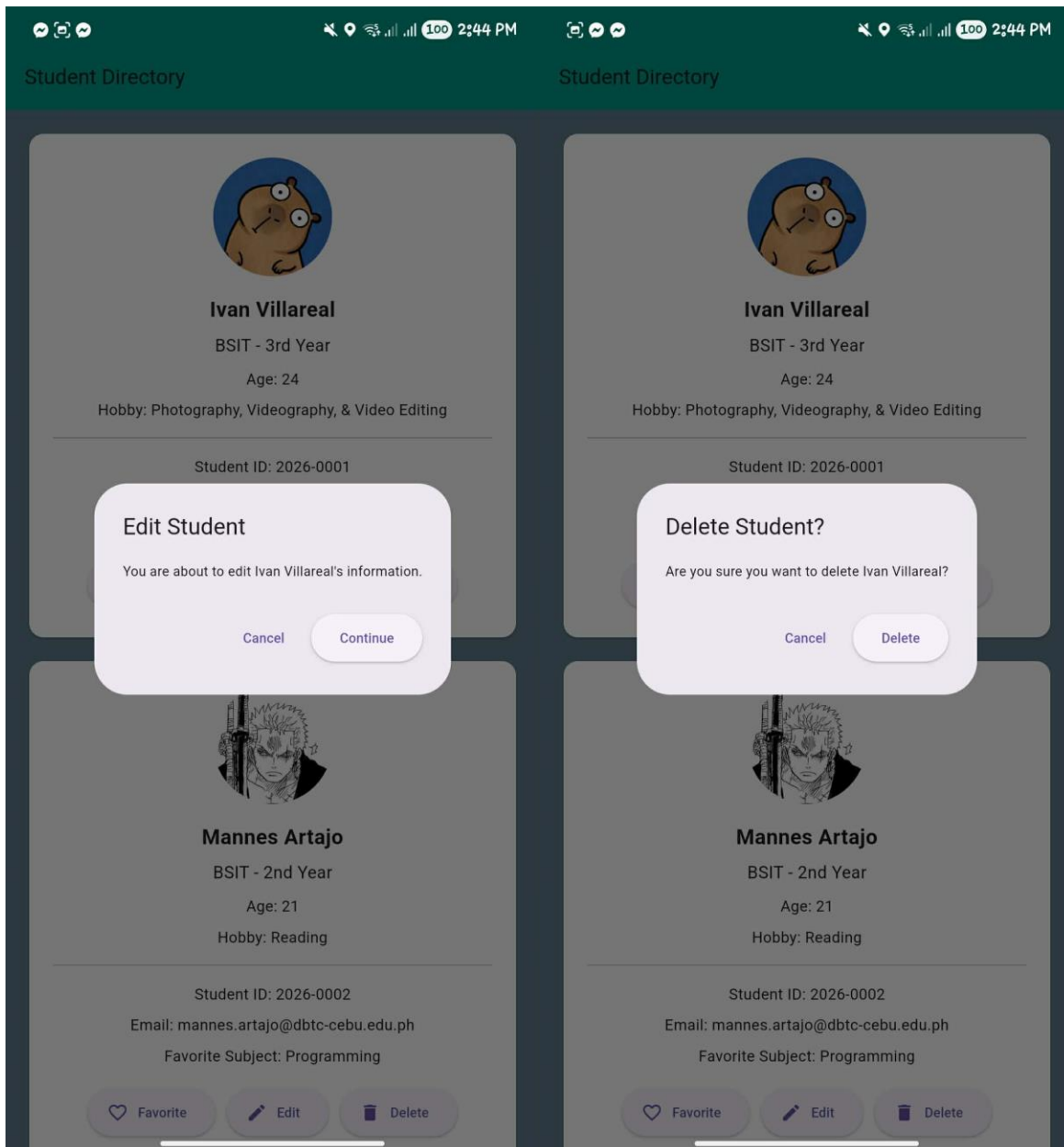
Your application should continue functioning correctly without affecting unrelated students.

Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:

- Multiple interactions working together
- Application still functioning normally



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
C:\Users\johnf\Downloads\ggs>git add .

C:\Users\johnf\Downloads\ggs>git commit -m "Feat: Student-interactions Completed"
[feature/student-interactions e1b5c2e] Feat: Student-interactions Completed
1 file changed, 255 insertions(+), 94 deletions(-)

C:\Users\johnf\Downloads\ggs>git push origin feature/student-interactions
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 2.34 KiB | 2.34 MiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'feature/student-interactions' on GitHub by visiting:
remote:   https://github.com/ivan013102/IT314AB_VILLAREAL/pull/new/feature/student-interactions
remote:
To https://github.com/ivan013102/IT314AB_VILLAREAL.git
 * [new branch]      feature/student-interactions -> feature/student-interactions
```